1. AR Camera Bridge (`ARRawCameraSource.cs`)

To feed the camera stream into the MediaPipe graph efficiently, I bypass the standard rendering pipeline and access the raw CPU image directly.

- **Raw Image Access**: Isubscribe to `ARCameraManager.frameReceived` to get the `XRCpuImage`.
- **Efficient Conversion**: Instead of reading pixels one by one (which is slow), I use Unity's `Unsafe` memory copying to convert the YUV/RGB image into a `Texture2D` (RGBA32) format.
- **Transformation**: The image is mirrored vertically (`MirrorY`) during conversion. This is critical because:
    - Computer Vision (MediaPipe) typically expects the image origin at the top-left.
    - Unity textures often originate at the bottom-left.
    - The mirroring ensures the hand tracking coordinates align correctly with the visual feed.

2. Coordinate Mapping (`HandColliderProxy.cs`)

MediaPipe returns hand landmarks in **Normalized Screen Coordinates** (0 to 1), where (0,0) is usually the top-left corner. Unity's World Space is 3D and metric-based. I map these two systems as follows:

The Pipeline

1. **Extraction**: I extract the **Index Finger Tip** (Landmark ID 8) from the `NormalizedLandmarkList`.
2. **Normalization Adjustment**:
    - `x_unity = x_mediapipe`
    - `y_unity = 1.0 - y_mediapipe` (Inverting Y to match Unity's Viewport coordinate system).

Projection (2D to 3D): I use the Main Camera (AR Camera) to project this 2D point into the 3D world: `Vector3 worldPos = mainCamera.ViewportToWorldPoint(new Vector3(x, y, interactionDepth));`

3.
    - `interactionDepth`: A fixed distance (e.g., 0.5 meters) from the camera. This defines the "plane" where interactions happen, effectively creating a 2D touch surface floating in front of the user.

3. Physics Interaction System

Directly moving a Transform often breaks physics collisions (objects pass through each other). To solve this, I use a **Physics Proxy**:

- **The Finger**: A simplified sphere with a `Rigidbody` and `Collider`.
- **Movement**: Instead of setting `transform.position`, Iuse `Rigidbody.MovePosition(worldPos)`. This allows the physics engine to calculate velocity and collision responses correctly.
- **Interaction**: Virtual objects (Buttons, Projectiles) respond to `OnTriggerEnter` or `OnCollisionEnter` events generated when this "Proxy Finger" penetrates their collider bounds.

4. Game Logic Bridge

- **Proxy -> Button**: The hand triggers a `ShootButton` or `ShieldButton`.
- **Button -> Game**: The button instantiates Prefabs (e.g., `RocketProjectile`).
- **Game -> Enemy**: Projectiles use standard Unity physics (`OnCollisionEnter`) to detect hits on the `GolemTarget`, verifying interactions via Layers and Tags to prevent self-collision.